
Recomendaciones generales

Esta guía contiene un Trabajo Práctico de MAAN II. El mismo deben ser resuelto íntegramente en **Python**. La resolución no es simple y se espera que resolverlo tome cierto tiempo por fuera de las clases.

Recuerden que pueden consultarlos sin ningún tipo de problema y que este trabajo y las siguientes son parte de la evaluación que tendrán de la materia. **Sean responsables, no se copien**. Si se detecta copia, toda la guía tendrá un 0 sin excepción. Se sugiere que **empiecen a resolver el TP con tiempo** de modo de tener tiempo para hacer consultas, no andar apurado a último momento.

El Trabajo Práctico debe resolverse de a grupos de **2 o 3 alumnos, no más y no menos**. Las consultas podrán realizarse por mail y en los momentos asignados en clase. Tener en cuenta que **si la consulta es realizada a último momento puede no ser factible responderla**.

kNN y predicción automática de precios de alojamientos

Estando ya sobre el final de la carrera y con cierto manejo en programación y ciencia de datos, estamos buscando una posición para tener una primera experiencia profesional en estos temas. Para ello, aplicamos a búsquedas de posiciones en el área de ciencia de datos de distintas empresas. Los procesos de selección suelen durar varias semanas, con entrevistas de distinto tipo y diferentes interlocutores. Entre ellas está lo que se suele denominar *entrevista técnica*, donde generalmente la empresa nos presenta un problema similar a los que se espera abordemos en la posición, pero con ciertas simplificaciones, y disponemos de una o dos semanas para entregar una solución al mismo. Tuvimos respuesta de *airbnb*, y nos piden armar un predictor simple para precios de alojamientos en base a datos históricos.¹

El problema

Nos piden programar un método para poder determinar de forma automática el precio de una nueva publicación para el marketplace basada en datos de otras publicaciones. Supongamos que contamos con un listado de publicaciones de nuestro sitio, los cuales ya tienen un precio determinado (mediante algún método) y ciertas características (ubicación, ratings, ambientes, etc.), tal como se muestra en la Figura 1.

Por otro lado, tenemos un usuario que desea ingresar a la plataforma una nueva publicación, y provee las características del alojamiento (Figura 2).

Qué precio deberíamos asignarle para que esté acorde a los restantes precios de la plataforma?

Los datos

¹Tomamos el problema como excusa para ejercitar programación implementando el método desde cero. Si fuese un escenario real, seguramente convenga directamente usar una implementación ya disponible en alguna librería el método en cuestión.

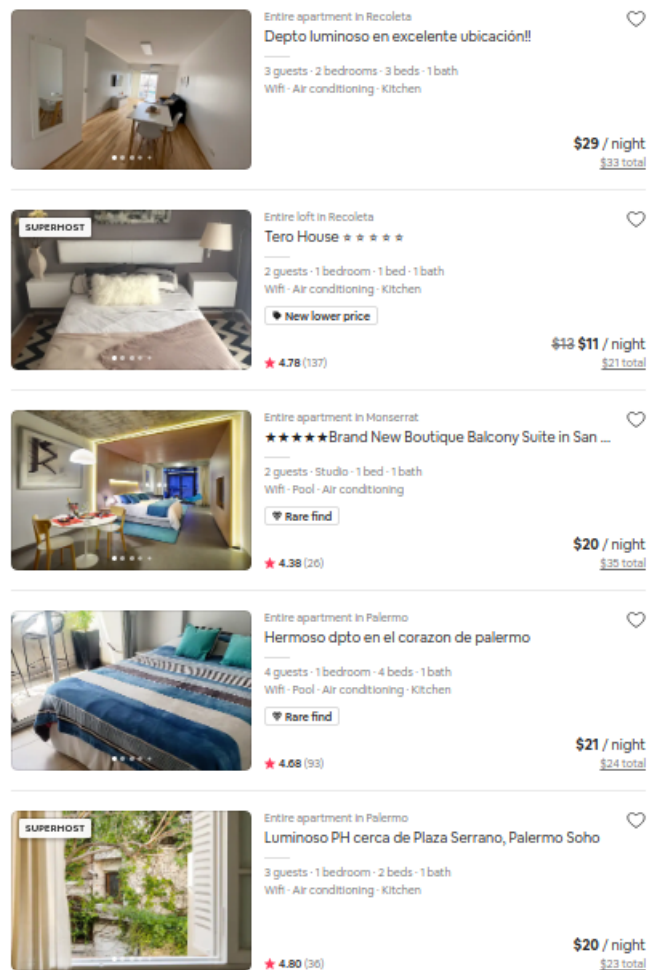


Figura 1: Extracto de publicaciones de airbnb con ejemplos de características

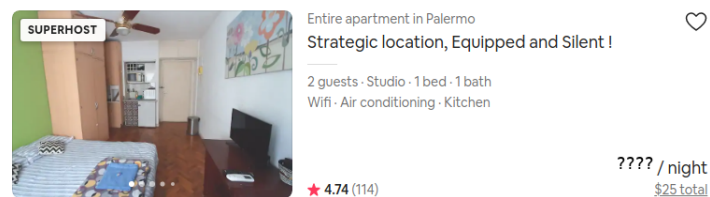


Figura 2: Nuevo aviso a ser publicado

Disponemos de dos bases, una de *training* con registros históricos y otra de *test* con los registros para los cuales debemos predecir el precio. Cada registro (fila) en la base representa un alojamiento. Los *features* (es decir las variables, columnas) son las siguientes:

- price: costo por noche de alojamiento.

- latitude: coordenada ubicación
- longitude: coordenada ubicación
- accomodates: cantidad máxima de personas habilitadas
- bedrooms: cantidad de habitaciones
- minimum_nights: cantidad mínima de noches para reservar
- number_or_reviews: cantidad de reviews
- review_scores_rating: evaluación general
- review_scores_location: evaluación general de la ubicación
- review_scores_value: evaluación respecto a la relación precio / calidad

La base de *test* no posee la columna precio, ya que es la variable que buscamos predecir. El resto de las variables se encuentran presentes en ambas bases y en el mismo orden.

Usamos dos bases reducidas de registros para ilustrar estas ideas. La base de training contenida en el archivo `training_small.csv` es la siguiente:

```
"price","latitude","longitude","accommodates","bathrooms","bedrooms","minimum_nights","number_of_reviews","review_scores_rating","review_scores_location","review_scores_value"
40,51.56802,-0.11121,2,1,1,1,21,97,9,9
75,51.48796,-0.16898,2,1,1,10,89,96,10,9
307,51.52195,-0.14094,6,2,3,4,42,94,10,9
29,51.57224,-0.20906,2,1,5,1,10,129,96,9,10
65,51.46507,-0.32421,2,1,1,1,6,90,10,8
195,51.47934,-0.28066,5,1,5,3,3,79,96,10,9
72,51.58461,-0.1617,2,0,1,2,528,97,10,10
80,51.53972,-0.05885,2,1,1,6,52,97,10,10
70,51.46871,-0.06455,2,1,1,2,55,96,9,10
```

y la base de test contenida en el archivo `testsmall.csv` es:

```
latitude,longitude,accommodates,bathrooms,bedrooms,minimum_nights,number_of_reviews,review_scores_rating,review_scores_location,review_scores_value
51.48756,-0.16915,2,1,1,7,1,100,10,10
51.51458,-0.17249,6,2,3,7,16,93,10,9
```

El modelo

Vamos a utilizar un modelo de regresión basado en el método de *vecinos más cercanos* (*k-Nearest Neighbors*, o *kNN*). Este es un método *no paramétrico* muy simple usado para problemas de clasificación y regresión en *machine learning*.

Resumidamente, el método asume disponible:

- Un conjunto de registros históricos (base de training)
- Una medida de distancia entre los datos. En el contexto del TP, vamos a usar la *distancia euclídea*. Dados $x = (x_1, \dots, x_n)$ y $\bar{x} = (\bar{x}_1, \dots, \bar{x}_n)$, la distancia entre x y $\bar{x}$ está dada por

$$\text{dist}(x, \bar{x}) = \|x - \bar{x}\|_2 = \sqrt{\sum_{i=1}^n (x_i - \bar{x}_i)^2}$$

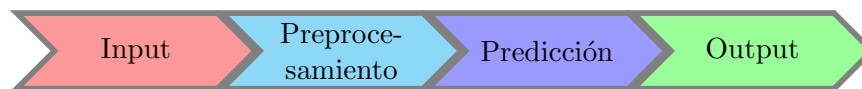
- Un valor k , indicando la cantidad de vecinos más cercanos a consultar.
- Uno o más registros cuya variable buscamos predecir (base de test).

Dada una nueva observación (registro en la base de test), el algoritmo realiza los siguientes pasos para predecir la variable en cuestión:

1. Se calcula la distancia a los registros de training, usando la medida de distancia.
2. Se identifican los k vecinos más cercanos acorde a esa distancia.
3. Promediamos la variable a predecir (en este caso, precio) de esos vecinos.

El trabajo

Este trabajo práctico va a estar dividido en cuatro etapas, simulando lo que podría ser un flujo posible al momento de abordar esta tarea en la práctica. Las etapas estimadas se muestran a continuación.



A continuación explicamos los objetivos y lineamientos a seguir en cada una de ellas.

1. Input y extracción de features (2.5 puntos).

- **Lectura de archivos (1 punto).** La función `file_to_matrix(filename)` recibe el nombre de un archivo csv, donde el separador es `,`, lo abre y convierte la información a una lista de listas, donde cada línea del archivo es una fila de la matriz. A modo de ejemplo, para el archivo `training_small.csv` el output debería ser:

```
[[40.0, 51.56802, -0.11121, 2.0, 1.0, 1.0, 1.0, 21.0, 97.0, 9.0, 9.0],
 [75.0, 51.48796, -0.16898, 2.0, 1.0, 1.0, 10.0, 89.0, 96.0, 10.0, 9.0],
 [307.0, 51.52195, -0.14094, 6.0, 2.0, 3.0, 4.0, 42.0, 94.0, 10.0, 9.0],
 [29.0, 51.57224, -0.20906, 2.0, 1.5, 1.0, 10.0, 129.0, 96.0, 9.0, 10.0],
 [65.0, 51.46507, -0.32421, 2.0, 1.0, 1.0, 1.0, 6.0, 90.0, 10.0, 8.0],
 [195.0, 51.47934, -0.28066, 5.0, 1.5, 3.0, 3.0, 79.0, 96.0, 10.0, 9.0],
 [72.0, 51.58461, -0.1617, 2.0, 0.0, 1.0, 2.0, 528.0, 97.0, 10.0, 10.0],
 [80.0, 51.53972, -0.05885, 2.0, 1.0, 1.0, 6.0, 52.0, 97.0, 10.0, 10.0],
 [70.0, 51.46871, -0.06455, 2.0, 1.0, 1.0, 2.0, 55.0, 96.0, 9.0, 10.0]]
```

- **Extracción de features (1 punto).** La función `get_features(dataset)` recibe una matriz de datos por filas con la base de training y devuelve la misma matriz pero sin la primera columna (es decir, quita el precio). A modo de ejemplo, para la base `training_small` el output debería ser

```
[[51.56802, -0.11121, 2.0, 1.0, 1.0, 1.0, 21.0, 97.0, 9.0, 9.0],
 [51.48796, -0.16898, 2.0, 1.0, 1.0, 10.0, 89.0, 96.0, 10.0, 9.0],
 [51.52195, -0.14094, 6.0, 2.0, 3.0, 4.0, 42.0, 94.0, 10.0, 9.0],
 [51.57224, -0.20906, 2.0, 1.5, 1.0, 10.0, 129.0, 96.0, 9.0, 10.0],
 [51.46507, -0.32421, 2.0, 1.0, 1.0, 1.0, 6.0, 90.0, 10.0, 8.0],
 [51.47934, -0.28066, 5.0, 1.5, 3.0, 3.0, 79.0, 96.0, 10.0, 9.0],
 [51.58461, -0.1617, 2.0, 0.0, 1.0, 2.0, 528.0, 97.0, 10.0, 10.0],
 [51.53972, -0.05885, 2.0, 1.0, 1.0, 6.0, 52.0, 97.0, 10.0, 10.0],
 [51.46871, -0.06455, 2.0, 1.0, 1.0, 2.0, 55.0, 96.0, 9.0, 10.0]]
```

- **Extracción de precio (0.5 puntos).** Análogamente, la función `get_y(dataset)` recibe la matriz de datos por filas con la base de training y devuelve una lista con la primera columna (el precio). A modo de ejemplo, para la base `training_small` el output debería ser:

```
[40.0, 75.0, 307.0, 29.0, 65.0, 195.0, 72.0, 80.0, 70.0]
```

Notar que la base de test tiene una estructura similar, pero alcanza simplemente con convertir el archivo csv a una matriz y no es necesario extraer features ni precio.

2. **Estandarización (0.5 puntos).** Este proceso se provee implementado casi en su totalidad en el módulo `preprocessing`. Solamente es necesario implementar la función `media`. Luego de ejecutar el proceso de estandarización, el resultado para la base `test_small` debería ser el siguiente (redondeado a 4 decimales)

```
[[-0.7459, -0.0028, -0.5276, -0.2156, -0.5345, 0.7921, -0.7281, 2.1549, 0.7071, 0.9999],  
 [-0.1404, -0.0420, 2.1859, 1.7253, 1.8708, 0.7921, -0.6290, -1.1562, 0.7071, -0.5000]]
```

3. **Predicción (5 puntos).** Esta etapa condensa la ejecución del algoritmo kNN para cada una de las observaciones. De más general a más específica, las funciones a implementar son las siguientes:

- **knn(X_training, X_test, y_training, k) (1 puntos):** Recorre los elementos de la base de test (filas de `X_test`), llama a la función `predict` para obtener la predicción y almacena los valores en una lista. *Función provista implementada parcialmente, pueden modificarlo si les resulta útil.* A modo de ejemplo, para la base `test_small` devuelve la siguiente lista:

```
[75.0, 192.33333333333334]
```

- **predict(X_training, y_training, new_obs, k) (1 puntos):** Dada un elemento de la base de test, predice el valor por noche. Para ello: (i) calcula la distancia euclídea a todas las observaciones del conjunto de training (función `get_distances`), (ii) se queda con las k de menor distancia (función `get_nearest_neighbors`), (iii) promedia los valores de `y` en training de las observaciones más cercanas y devuelve ese valor.

- **get_distances(target, X, y_training) (1 puntos):** Recibe un elemento de la base de test (`target`), la base de training (`X`, con n elementos) y la lista de precios. Devuelve una lista de n tuplas donde cada una contiene: (i) la distancia euclídea entre la i -ésima observación de la base de training y `target`; (ii) el precio asociado a la i -ésima observación de training, (iii) el índice de la i -ésima observación (i). A modo de ejemplo, el vector resultado para los registros en la base `test_small` son:

(3.9586057038293707, 40.0, 0),	(5.488658471372138, 40.0, 0),
(2.6386124755845595, 75.0, 1),	(4.507195672166508, 75.0, 1),
(5.365903693562624, 307.0, 2),	(1.1007553805923052, 307.0, 2),
(3.7882761486260352, 29.0, 3),	(5.104892664823618, 29.0, 3),
(6.17387197571091, 65.0, 4),	(5.36290256099111, 65.0, 4),
(4.485808219731925, 195.0, 5),	(2.6897959750897984, 195.0, 5),
(4.984465393475517, 72.0, 6),	(7.083374610335392, 72.0, 6),
(2.2926620217504583, 80.0, 7),	(4.998660071352363, 80.0, 7),
(3.478169109471362, 70.0, 8)]	(5.5329097274926164, 70.0, 8)]

- `euclidean_distance(x, y)`: (0.5 puntos): dados dos vectores (listas) de igual cantidad de elementos, devuelve la distancia euclídea entre ambos (definida previamente en el enunciado).
- `get_nearest_neighbors(distancias, k)` (1.5 puntos): Toma como input la estructura devuelta por `get_distances` y elige los k más cercanos en base a la distancia. A modo de ejemplo, tomando $k = 3$ obtenemos los siguientes resultados para los elementos de la base `test_small`:

<code>[(2.2926620217504583, 80.0, 7),</code>	<code>[(1.1007553805923052, 307.0, 2),</code>
<code>(2.6386124755845595, 75.0, 1),</code>	<code>(2.6897959750897984, 195.0, 5),</code>
<code>(3.478169109471362, 70.0, 8)]</code>	<code>(4.507195672166508, 75.0, 1)]</code>

4. **Output de predicción (1 punto)**. La función `write_predictions(preds, filename)` debe abrir un archivo con nombre `filename`, escribir la lista `preds` (con las predicciones para cada registro del conjunto de test) y luego cerrarlo. La primera línea es el header (prediccion), luego una prediccion por línea. A modo de ejemplo, para la base `test_small` el archivo debería contener:

```
prediccion
75.0
192.33333333333334
```

5. **Experimentación y submit de solución (1 punto)**. Finalmente, se deberá ejecutar el método para la base estándar que contiene mayor cantidad de registros tanto para training como para test. Para evaluar la calidad del resultado, el mismo deberá ser subido a una competencia (privada, organizada por la cátedra) de la plataforma Kaggle.

Modalidad de entrega

Además del código con las implementaciones es posible entregar un mini informe de no más de tres carillas en donde detallen, en caso de considerarlo necesario, decisiones que tomaron para resolver uno o más ejercicios y que ayuden a entender la estrategia de resolución.

Fechas de entrega

Formato Electrónico: **miércoles 21 de Abril, hasta las 23:59 hs.**, enviando el trabajo (informe + código) a las direcciones `maan2utdt@gmail.com`. El subject del email debe comenzar con el texto [TP2 MAAN II] seguido de la lista de apellidos de los integrantes del grupo. Todos los integrantes del grupo deben estar copiados en el email.

Importante: El horario es estricto. Los correos recibidos después de la hora indicada serán considerados re-entrega.